



RAMAIAH
Institute of Technology

**RAMAIAH INSTITUTE OF TECHNOLOGY
BANGALORE – 560054
(Autonomous Institute, Affiliated to VTU)**

**Department of Computer Science &
Engineering**

**20-Mark Component Report
On**

**Comparative Analysis of Dijkstra and A* Algorithms for Determining
the Shortest Route**

CS43: Design and Analysis of Algorithms

Varun U Pratap	1MS24CS215
Utsav S R	1MS24CS208
Suhas Vankadari	1MS24CS191
Shrey Kumar Singh	1MS24CS175

**Under the Guidance
Pradeep Kumar D**

**Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
MSR Nagar, MSRIT Post, Bangalore-560054**

Sept – Dec 2025



RAMAIAH INSTITUTE OF TECHNOLOGY, BANGALORE – 560054
(Autonomous Institute, Affiliated to VTU)

Department of Computer Science & Engineering

Evaluation Report

**Title of the Topic: Comparative Analysis of Dijkstra and A* Algorithms for
Determining the Shortest Route**

Team Member Details		
Sl. No.	USN	Name
1.	1MS24CS215	Varun U Pratap
2.	1MS24CS208	Utsav S R
3.	1MS24CS191	Suhas Vankadari
4.	1MS24CS175	Shrey Kumar Singh

SL No.	Component	Maximum Marks	Marks Obtained
1.			
2.			
3.			
4.			
Total Marks		20	

Signature of the Student

Signature of the Faculty

Signature of Head of the Department

TABLE OF CONTENTS

Sl No.	Title	Page No
	Abstract	1 – 1
1.	Introduction	2 – 4
2.	Background and Theory	4 – 10
3.	Problem Description and Graph Model	10 – 12
4.	Java Implementation	12 – 25
5.	Results	25 – 27
6.	Improvements over the Original Paper	27 – 29
7.	Discussion	30 – 31
8.	Conclusion	32 – 33
	References	33 – 34

Abstract

Shortest path determination is a fundamental problem in computer science with broad applications in navigation systems, geographic information systems (GIS), logistics, robotics, and urban planning. This report presents a comprehensive implementation and comparative analysis of two widely studied shortest path algorithms — Dijkstra's algorithm and the A* (A-Star) algorithm — based on the research conducted by Ardiansyah, Abdul Muin Nasution, and Muhammad Iqbal (2025), published in Bit-Tech: Binary Digital Technology, Vol. 8, No. 2. The original study modeled a real-world urban road network in Medan, Indonesia, as a weighted directed graph, using distance data obtained from Google Maps, and compared both algorithms on a seven-node graph spanning from SMKN 9 Medan (Node A) to Gramedia Gajah Mada (Node G) across three candidate routes.

While the original paper implemented the algorithms in Python as a single procedural script, this report presents a complete re-implementation in Java using an object-oriented design, comprising five distinct classes: Graph, PathResult, DijkstraAlgorithm, AStarAlgorithm, and Main. This architectural improvement enhances modularity, readability, reusability, and scalability compared to the original flat-script approach.

The implemented algorithms were applied to the identical graph data from the paper, including the same edge weights and heuristic values, to ensure a fair and reproducible comparison. Both algorithms successfully identified the same optimal route — $A \rightarrow B \rightarrow E \rightarrow G$ — with a total distance of 5.7 km, confirming the theoretical property that both Dijkstra and A* guarantee optimality when applied to graphs with non-negative edge weights and an admissible heuristic function. However, a measurable difference was observed in computational efficiency: the A* algorithm explored fewer nodes than Dijkstra's algorithm, demonstrating the practical advantage of heuristic-guided search in reducing unnecessary node expansions.

Beyond replication, the Java implementation introduced several technical improvements over the original Python script, including deterministic node ordering through alphabetical sorting, explicit g-score tracking in the A* implementation for correctness under non-consistent heuristics, and a dedicated PathResult class that cleanly encapsulates algorithm output for extensibility. These enhancements make the implementation better suited for integration into larger systems.

This report discusses the theoretical foundations of both algorithms, describes the graph model and data used, walks through the implementation in detail with annotated code snippets, presents and analyzes the results, and reflects on the improvements made. The findings corroborate the conclusions of the original paper: while Dijkstra's algorithm remains a reliable and universally applicable solution for small graphs, A* offers superior computational efficiency for real-world navigation applications, making it the preferred choice in systems where speed and scalability are critical.

1. Introduction

1.1 Background and Motivation

In an increasingly connected and fast-paced world, the ability to determine the most efficient route between two locations has become a critical requirement across a wide range of domains. From everyday navigation applications such as Google Maps and Waze, to industrial systems managing logistics fleets, emergency response coordination, robotic path planning, and smart city infrastructure, shortest path algorithms form the computational backbone of modern routing and navigation technology. The efficiency, accuracy, and scalability of these algorithms directly influence the quality of decisions made by both automated systems and human users.

At the heart of these systems lies a well-studied problem in computer science and graph theory: given a weighted graph representing a network of locations and connections, find the path between two nodes that minimizes the total accumulated cost, whether that cost represents distance, time, fuel consumption, or any other quantifiable metric. This problem, known as the Single-Source Shortest Path (SSSP) problem, has been the subject of extensive research for decades, yielding a rich collection of algorithms that vary in their approach, efficiency, and applicability.

Among the many algorithms developed to address this problem, two stand out as particularly prominent and widely used in both academic literature and practical systems: Dijkstra's algorithm, introduced by Edsger W. Dijkstra in 1959, and the A* (A-Star) algorithm, developed by Peter Hart, Nils Nilsson, and Bertram Raphael in 1968. Dijkstra's algorithm is an uninformed, greedy search method that guarantees finding the globally optimal shortest path by systematically exploring all reachable nodes in order of increasing cumulative distance. The A* algorithm, in contrast, is an informed search method that incorporates a heuristic function to estimate the remaining cost from any given node to the destination, allowing it to prioritize more promising paths and reduce the number of nodes that need to be explored.

Despite the well-established theoretical superiority of A* in terms of search efficiency, empirical comparisons of the two algorithms on real-world road networks remain valuable and relevant. The practical performance of A* depends heavily on the quality and accuracy of its heuristic function, and in certain graph configurations, the efficiency gains may be marginal or even absent. Understanding when and why one algorithm outperforms the other, and under what conditions their results converge, is therefore of both theoretical interest and practical importance.

1.2 The Original Research Paper

This report is based on the research paper titled "Comparative Analysis of Dijkstra and A* Algorithms for Determining the Shortest Route," authored by Ardiansyah, Abdul Muin Nasution, and Muhammad Iqbal, and published in Bit-Tech: Binary Digital Technology, Vol. 8, No. 2, December 2025. The paper presents a focused empirical study comparing Dijkstra and A* on a real-world urban road network in Medan, North Sumatra, Indonesia, modeled as a weighted directed graph of seven nodes and eleven edges, with distances derived from Google Maps.

The study selected SMKN 9 Medan, a vocational high school located at Jl. Patriot No. 20A, Lalang, Medan Sunggal, as the starting node (Node A), and the Gramedia Gajah Mada bookstore, located at Jl. Gajah Mada No. 23, Petisah Hulu, Medan Baru, as the destination node (Node G). Five intermediate nodes — representing real landmarks including a government immigration office, a commercial building, a bank, a convenience store, and a hospital — were used to construct the graph. The heuristic values used in the A* algorithm were straight-line distance estimates from each node directly to the destination, also obtained from Google Maps.

The original implementation was written in Python as a single procedural script, producing console output that compared the two algorithms across three metrics: the shortest path found, the total distance traveled, and the number of nodes explored during the search. The paper concluded that both algorithms identified the same optimal route ($A \rightarrow B \rightarrow E \rightarrow G$, 5.7 km), but that A* was more computationally efficient, exploring fewer nodes and requiring less processing time.

1.3 Objectives of This Report

The primary objective of this report is to replicate the findings of Ardiansyah et al. (2025) through a complete re-implementation of both algorithms in Java, using an object-oriented architecture that improves upon the original Python implementation in terms of design, correctness, and extensibility. Specifically, this report aims to:

- Faithfully reproduce the graph model, edge weights, heuristic values, and experimental conditions described in the original paper, so that the results can be directly compared and validated.
- Implement Dijkstra's algorithm and the A* algorithm in Java using clean, well-documented, object-oriented code, organized across multiple classes with clearly defined responsibilities.
- Verify that both algorithms produce the same optimal path and total distance as reported in the paper, confirming both the correctness of the implementation and the theoretical properties of the algorithms.
- Identify and articulate the specific technical improvements made in the Java implementation relative to the original Python script, including improvements in determinism, result encapsulation, and code organization.
- Provide a thorough discussion of the theoretical foundations of both algorithms, the design decisions made during implementation, and the broader implications of the results for real-world navigation system design.

1.4 Scope and Limitations

This report focuses exclusively on the static, directed, weighted graph described in the original paper. The graph consists of seven nodes and eleven directed edges, with fixed distance values that do not change during the execution of the algorithms. Dynamic factors such as real-time traffic conditions, road closures, construction, signal delays, and time-of-day variation are not considered. The heuristic values used in the A* algorithm are straight-line distance estimates and are assumed to be admissible — that is, they never overestimate the true remaining cost to the goal — which is a necessary condition for A* to guarantee an optimal solution.

Additionally, while the Java implementation is designed with scalability in mind, the experimental results presented in this report are specific to the seven-node Medan road network. Performance characteristics observed here, such as the difference in nodes explored between Dijkstra and A*, may not generalize directly to larger or more complex graphs. The report does, however, discuss how the implementation could be extended and what behavior might be expected at larger scales.

2. Background and Theory

2.1 Graph Theory Fundamentals

Before examining the algorithms themselves, it is important to establish the mathematical foundation upon which they operate. Both Dijkstra's algorithm and the A* algorithm are designed to operate on graphs — abstract mathematical structures used to model pairwise relationships between objects. Formally, a graph G is defined as an ordered pair $G = (V, E)$, where V is a finite set of vertices (also called nodes) representing locations or entities, and E is a set of edges representing connections or relationships between pairs of vertices.

In the context of road network modeling, vertices correspond to physical locations such as intersections, landmarks, or points of interest, while edges correspond to roads or paths connecting those locations. A weighted graph assigns a numerical value, called a weight or cost, to each edge. In routing applications, this weight typically represents the distance, travel time, or fuel cost associated with traversing the corresponding road segment. A directed graph (or digraph) further specifies that each edge has a direction, meaning that traversal is only permitted from one endpoint to the other in the specified direction, which is appropriate for modeling one-way roads or asymmetric travel costs.

The graph used in this study is a directed weighted graph, where edge weights represent road distances in kilometers as measured by Google Maps. The goal of both algorithms is to find, within this graph, the path from a designated source vertex to a designated destination vertex such that the sum of edge weights along the path is minimized. This path is called the shortest path, and its total weight is called the shortest path distance.

A path in a graph is formally defined as a sequence of vertices $v_0, v_1, v_2, \dots, v_k$ such that for each consecutive pair (v_i, v_{i+1}) , there exists a directed edge from v_i to v_{i+1} in E . The cost of the path is the sum of the weights of all edges traversed:

$$\text{cost}(\text{path}) = \sum w(v_i, v_{i+1}) \text{ for } i = 0 \text{ to } k-1$$

where $w(u, v)$ denotes the weight of the directed edge from vertex u to vertex v . The shortest path problem asks for the path with the minimum such cost among all paths from the source to the destination.

2.2 Dijkstra's Algorithm

2.2.1 Overview and History

Dijkstra's algorithm was conceived by Dutch computer scientist Edsger W. Dijkstra in 1956 and published in 1959. It is one of the most celebrated and widely studied algorithms in computer science, and remains a foundational topic in undergraduate and graduate curricula worldwide. The algorithm solves the single-source shortest path problem for graphs with non-negative edge weights, meaning it finds the shortest path from a single source node to all other nodes in the graph simultaneously, though in practice it is often terminated early once the destination node is reached.

The algorithm belongs to the category of greedy algorithms — it makes locally optimal decisions at each step with the goal of finding a globally optimal solution. Its correctness relies critically on the assumption that all edge weights are non-negative, which ensures that once a node has been finalized (marked as visited), its shortest distance cannot later be improved by discovering a new path through a subsequently explored node.

2.2.2 Algorithm Description

Dijkstra's algorithm maintains a set of visited nodes and a priority queue of candidate nodes ordered by their current best-known distance from the source. Initially, the source node is assigned a distance of zero, and all other nodes are assigned a distance of infinity, reflecting the fact that no path to them has yet been discovered. The algorithm then proceeds iteratively:

At each iteration, the node u with the smallest tentative distance is dequeued from the priority queue and marked as visited, meaning its shortest distance has been finalized and will not change. For each unvisited neighbor v of u , the algorithm performs a relaxation step: it computes the candidate distance $d(u) + w(u, v)$ and compares it to the current best-known distance $d(v)$. If the candidate distance is smaller, $d(v)$ is updated and v is re-inserted into the priority queue with its new priority. This process continues until the destination node is dequeued and finalized, at which point the shortest path can be reconstructed by tracing back the predecessor pointers from the destination to the source.

The core relaxation formula is:

$$d(v) = \min(d(v), d(u) + w(u, v))$$

This formula expresses the fundamental principle of dynamic programming underlying Dijkstra's algorithm: the shortest path to v either already exists, or it passes through u and then takes the edge (u, v) .

2.2.3 Pseudocode

```
DIJKSTRA(Graph G, Node start, Node goal):  
    for each node v in G:  
        dist[v] ← ∞  
        previous[v] ← null
```



```

    visited ← empty set

    dist[start] ← 0
    PQ ← priority queue ordered by dist
    PQ.insert(start, priority=0)

    while PQ is not empty:
        u ← PQ.extractMin()
        if u is in visited: continue
        visited.add(u)

        if u == goal:
            return reconstructPath(previous, start, goal), dist[goal]

        for each neighbor v of u with edge weight w:
            if v not in visited:
                candidate ← dist[u] + w
                if candidate < dist[v]:
                    dist[v] ← candidate
                    previous[v] ← u
                    PQ.insert(v, priority=candidate)

    return NO PATH FOUND

```

Pseudocode 1: Dijkstra's Algorithm

2.2.4 Correctness and Complexity

The correctness of Dijkstra's algorithm follows from the greedy choice property: since all edge weights are non-negative, the node with the smallest tentative distance at any point in the algorithm cannot have its distance reduced by any future exploration. This means that when a node is dequeued, its distance is already optimal and can be safely finalized.

The time complexity of Dijkstra's algorithm depends on the data structure used for the priority queue. With a simple binary heap, the complexity is $O((V + E) \log V)$, where V is the number of vertices and E is the number of edges. With a more sophisticated Fibonacci heap, this can be reduced to $O(E + V \log V)$, though Fibonacci heaps carry high constant factors and are rarely used in practice. For dense graphs where $E \approx V^2$, the algorithm becomes $O(V^2 \log V)$, which can be expensive for large networks. The space complexity is $O(V)$ for the distance and predecessor arrays, plus $O(V)$ for the priority queue.

2.3 The A* Algorithm

2.3.1 Overview and History

The A* algorithm was introduced in 1968 by Peter Hart, Nils Nilsson, and Bertram Raphael at the Stanford Research Institute, in the context of research on robotic path planning. It was originally presented as a generalization of Dijkstra's algorithm that incorporates domain-specific knowledge about the problem in the form of a heuristic function, allowing the search to be directed toward the goal rather than expanding uniformly in all directions. Over the decades since its introduction, A* has become one of the most widely used algorithms in artificial intelligence, game development, robotics, and geographic information systems.

The key insight behind A* is that Dijkstra's algorithm, while correct, is wasteful: it explores nodes in all directions regardless of whether they are likely to lead toward the goal. In a large road network, this means potentially exploring thousands of nodes that are geographically far from the destination before eventually finding the optimal path. A* addresses this by augmenting the priority of each node with an estimate of how far it still is from the goal, effectively biasing the search toward nodes that are both close to the start and close to the destination.

2.3.2 The Evaluation Function

The central innovation of A* is its evaluation function $f(n)$, defined as:

$$f(n) = g(n) + h(n)$$

where:

- **$g(n)$** is the exact cumulative cost of the best known path from the start node to node n . This is identical to the distance value maintained by Dijkstra's algorithm and represents what has already been paid to reach n .
- **$h(n)$** is the heuristic estimate of the remaining cost from node n to the goal. This value is not computed from the graph but rather estimated using some domain-specific knowledge, such as the straight-line (Euclidean) distance between n and the goal in a geographic routing problem.
- **$f(n)$** is the estimated total cost of the cheapest path from the start to the goal that passes through node n . It represents the algorithm's best guess at how expensive the overall path through n will be.

By ordering the priority queue according to $f(n)$ rather than $g(n)$ alone, A* preferentially expands nodes that appear to offer the most promising overall paths, rather than simply nodes that are closest to the start. This allows it to reach the goal while exploring far fewer nodes than Dijkstra's algorithm in most practical scenarios.

2.3.3 Admissibility and Consistency of the Heuristic

The theoretical guarantees of A* depend critically on properties of the heuristic function $h(n)$. Two properties are of particular importance:

Admissibility: A heuristic $h(n)$ is said to be admissible if it never overestimates the true cost to reach the goal from node n . Formally:

$$h(n) \leq h^*(n) \text{ for all nodes } n$$

where $h^*(n)$ is the true optimal cost from n to the goal. An admissible heuristic ensures that A* will always find an optimal solution, because it never causes the algorithm to prematurely abandon a path that might turn out to be optimal.

Consistency (Monotonicity): A heuristic $h(n)$ is consistent if for every node n and every neighbor n' of n reached via an edge of cost $w(n, n')$:

$$h(n) \leq w(n, n') + h(n')$$

This is analogous to the triangle inequality in geometry. A consistent heuristic is always admissible, and it additionally guarantees that once a node is added to the closed set (visited), its g -score is already optimal and will never need to be updated. This property simplifies the implementation of A* and ensures that nodes do not need to be re-opened once closed.

In this study, the heuristic values used are straight-line distance estimates from each node to the destination, obtained from Google Maps. Straight-line distance is a classic admissible heuristic for geographic routing problems, because the straight-line distance between two points is always less than or equal to the actual road distance, which must follow the road network and cannot be shorter than a straight line.

2.3.4 Pseudocode

```
A_STAR(Graph G, Node start, Node goal, Heuristic h):  
    for each node v in G:  
        g[v] ← ∞  
        previous[v] ← null  
        visited ← empty set  
  
    g[start] ← 0  
    f[start] ← g[start] + h(start)  
    openSet ← priority queue ordered by f  
    openSet.insert(start, priority=f[start])  
  
    while openSet is not empty:  
        u ← openSet.extractMin()  
        if u is in visited: continue  
        visited.add(u)  
  
        if u == goal:  
            return reconstructPath(previous, start, goal), g[goal]
```

```

    for each neighbor v of u with edge weight w:
        if v not in visited:
            tentative_g ← g[u] + w
            if tentative_g < g[v]:
                g[v] ← tentative_g
                previous[v] ← u
                f[v] ← g[v] + h(v)
                openSet.insert(v, priority=f[v])

return NO PATH FOUND

```

Pseudocode 2: A Algorithm*

2.3.5 Correctness and Complexity

When the heuristic h is admissible, A* is guaranteed to find an optimal path. When h is also consistent, A* is guaranteed to expand each node at most once, making the algorithm maximally efficient for the given heuristic. The time and space complexity of A* is $O(b^d)$ in the worst case, where b is the branching factor of the graph and d is the depth of the optimal solution. However, in practice — particularly with a good heuristic — A* explores significantly fewer nodes than this worst case suggests, and far fewer than Dijkstra's algorithm explores on the same graph.

The quality of the heuristic directly determines how much of the search space A* can prune. A perfect heuristic ($h(n) = h^*(n)$ for all n) would allow A* to follow the optimal path directly without exploring any unnecessary nodes. A heuristic of $h(n) = 0$ for all n reduces A* to exactly Dijkstra's algorithm, exploring the entire reachable graph. In practice, good heuristics fall somewhere between these extremes, providing meaningful but imperfect guidance.

2.4 Comparison of the Two Algorithms

The fundamental difference between Dijkstra's algorithm and A* lies in the information used to prioritize node exploration. Dijkstra uses only backward-looking information — how far we have already traveled to reach a node — while A* combines this with forward-looking information — an estimate of how far we still need to travel to reach the goal. This distinction has important consequences for efficiency, particularly in large graphs where many nodes are far from the optimal path and need not be explored at all.

Both algorithms share the same path reconstruction mechanism, the same relaxation condition for updating distances, and the same guarantee of optimality under their respective assumptions. They differ in their priority function, and consequently in the order in which nodes are explored and the total number of nodes that must be examined before the goal is reached.

The table below summarizes the key theoretical differences between the two algorithms:

Property	Dijkstra's Algorithm	A* Algorithm
Search Type	Uninformed (Blind)	Informed (Heuristic)
Priority Function	$g(n)$ only	$f(n) = g(n) + h(n)$
Heuristic Required	No	Yes (admissible)
Optimality Guaranteed	Yes (non-negative weights)	Yes (admissible heuristic)
Nodes Explored	More (all directions)	Fewer (directed to goal)
Time Complexity	$O((V+E) \log V)$	$O(b^d)$ worst case
Best Use Case	Small graphs	Large graphs, good heuristic

Table 2.1: Theoretical Comparison of Dijkstra and A* Algorithms

2.5 Heuristic Design for Geographic Routing

In geographic routing problems, the most commonly used heuristic for A* is the straight-line or Euclidean distance between the current node and the goal. This heuristic is admissible because the straight-line distance is always a lower bound on the actual road distance: no road can be shorter than the straight line between two points. It is also easy to compute given the geographic coordinates of the nodes.

In the study by Ardiansyah et al. (2025), the heuristic values were obtained by measuring direct distances from each node to the destination using Google Maps, rather than computing them from GPS coordinates. While this approach produces slightly different values than pure Euclidean distance calculation, the resulting heuristic values remain admissible as long as the straight-line measurements do not exceed the actual road distances, which is guaranteed by the geometry of the problem. The heuristic values used in this study range from 0.0 km at the goal node (Node G) to 4.7 km at the start node (Node A), reflecting the decreasing proximity of each node to the destination.

3. Problem Description and Graph Model

3.1 Case Study Overview

The case study used in Ardiansyah et al. (2025), and replicated in this implementation, involves finding the shortest driving route between two real locations in Medan City, North Sumatra, Indonesia. The starting point is SMKN 9 Medan (Sekolah Menengah Kejuruan Negeri 9 Medan), a state vocational high school located at Jl. Patriot No. 20A, Lalang, Kecamatan Medan Sunggal, and the destination is Gramedia Gajah Mada, a major bookstore chain located at Jl. Gajah Mada No. 23, Petisah Hulu, Kecamatan Medan Baru. Both locations are situated within Medan City and are connected through a network of urban roads that pass through several intermediate landmarks.

The road network between these two locations was modeled as a directed weighted graph consisting of seven nodes (A through G) and eleven directed edges. Each node corresponds to a real-world landmark, and each directed edge represents a one-way road segment connecting two landmarks, with a weight equal to the road distance in kilometers as measured by Google Maps. The graph captures three alternative routes from the

start to the destination, allowing both algorithms to evaluate and select the globally optimal path.

3.2 Node Definitions

The seven nodes of the graph and their corresponding real-world locations are described in the following table, as originally presented in Ardiansyah et al. (2025):

Node	Short Name	Full Location Name and Address
A	SMKN 9	Sekolah Menengah Kejuruan Negeri 9 Medan, Jl. Patriot No.20A, Lalang, Kec. Medan Sunggal, Kota Medan, Sumatera Utara 20123 [START]
B	Imigrasi	Kantor Imigrasi Kelas 1 Khusus TPI Medan, Jalan Gatot Subroto KM.6,2 No.268A, Sei Sikambing C. II, Kec. Medan Helvetia, Kota Medan
C	MICC	Gedung MICC, Jalan Gagak Hitam, Sei Sikambing B, Kota Medan, Sumatera Utara
D	Muamalat	Bank Muamalat KCU Medan Baru, Jl. Iskandar Muda No.121, Sei Sikambing D, Kec. Medan Petisah, Kota Medan 20119
E	Indomaret	Indomaret Darussalam Bhakti, Jl. Darussalam No.15, Sei Sikambing D, Kec. Medan Petisah, Kota Medan 20113
F	RS. Bunda	RSU Bunda Thamrin, Jl. Sei Batang Hari No.28-30-42, Babura Sunggal, Kec. Medan Sunggal, Kota Medan 20112
G	Gramedia	Gramedia Gajah Mada Medan, Jl. Gajah Mada No.23, Petisah Hulu, Kec. Medan Baru, Kota Medan 20152 [DESTINATION]

Table 3.1: Node Definitions — Locations in the Medan Road Network

3.3 Edge Weights — Road Distances $g(n)$

The eleven directed edges of the graph and their associated weights (road distances in kilometers from Google Maps) are presented in the table below. These distances form the basis of the cumulative path cost $g(n)$ used by both algorithms:

From	To	Distance $g(n)$ (km)
A	B	1.2
A	C	1.2
B	D	4.0
B	E	2.8
B	F	3.3
C	F	3.3
D	G	0.6
E	D	1.5
E	G	1.7

From	To	Distance g(n) (km)
E	F	0.7
F	G	1.5

Table 3.2: Edge Weights — Road Distances Between Nodes (Source: Google Maps)

3.4 Heuristic Values h(n)

The A* algorithm requires a heuristic estimate $h(n)$ representing the straight-line distance from each node directly to the destination node G (Gamedia Gajah Mada). These values were obtained from Google Maps by measuring direct distances and serve as the admissible heuristic guiding A*'s search. Note that $h(G) = 0.0$ since the goal node has no remaining distance to travel:

Node	Route to G	h(n) (km)
A	A-G	4.7
B	B-G	3.9
C	C-G	4.1
D	D-G	0.4
E	E-G	1.3
F	F-G	1.4
G	G-G	0.0

Table 3.3: Heuristic Values h(n) — Straight-Line Distance to Goal (Source: Google Maps)

3.5 Graph Structure and Candidate Routes

Based on the edge data, the directed weighted graph has eleven edges forming three primary candidate routes from Node A to Node G. These routes are listed below along with their total distances, computed by summing the weights of the edges traversed:

1. **Route 1: A → B → E → G | $1.2 + 2.8 + 1.7 = 5.7$ km (OPTIMAL)**
2. Route 2: A → B → D → G | $1.2 + 4.0 + 0.6 = 5.8$ km
3. Route 3: A → B → F → G | $1.2 + 3.3 + 1.5 = 6.0$ km / A → C → F → G | $1.2 + 3.3 + 1.5 = 6.0$ km

There are also sub-routes passing through E that connect to D or F before reaching G, but as the algorithm evaluations show, none of these produce a shorter cumulative distance than Route 1. Both Dijkstra and A* must evaluate these alternatives during their execution, and both ultimately confirm A → B → E → G as the optimal path at 5.7 km.

4. Java Implementation

4.1 Architecture Overview

The Java implementation follows a clean object-oriented architecture distributed across five separate classes, each with a single, well-defined responsibility. This stands in contrast to the original Python implementation

in Ardiansyah et al. (2025), which was written as a single flat procedural script. The five classes and their roles are as follows:

- `Graph.java` — Represents the weighted directed graph. Manages the adjacency list, edge definitions, and heuristic values. Provides methods for adding nodes, adding edges, setting heuristics, and querying neighbors.
- `PathResult.java` — Encapsulates the output of a shortest-path algorithm execution. Stores the found path, total distance, and number of nodes explored. Provides formatted output methods.
- `DijkstraAlgorithm.java` — Implements Dijkstra's algorithm as a static utility class. Takes a `Graph` object and start/goal node identifiers, returns a `PathResult`.
- `AStarAlgorithm.java` — Implements the A* algorithm as a static utility class. Uses heuristic values from the `Graph` object to compute $f(n) = g(n) + h(n)$, returns a `PathResult`.
- `Main.java` — Entry point of the program. Constructs the graph, sets heuristics, executes both algorithms, and prints a formatted comparison table.

This separation of concerns makes the codebase modular and extensible. Adding a new algorithm, a new graph, or a new heuristic requires modifying only the relevant class without touching the others. The `Graph` class and `PathResult` class in particular are fully reusable across any shortest-path problem, not just the Medan case study. Sections 4.2 through 4.6 discuss the design and key decisions of each class with annotated excerpts; the complete, unabridged source code for all five classes is provided in Section 4.7.

4.2 `Graph.java` — Graph Data Structure

The `Graph` class is the foundation of the entire implementation. It uses a `HashMap`-based adjacency list to represent the directed weighted graph, where each key is a node identifier (a `String` such as "A" or "G") and each value is a list of `Edge` objects representing outgoing connections. An inner static class `Edge` holds the destination node name and the edge weight (distance in km).

The class also maintains a separate `HashMap` for heuristic values, mapping each node identifier to its $h(n)$ value. This design choice — storing heuristic data inside the `Graph` rather than in the algorithm class — makes sense because the heuristic is a property of the problem domain (the geographic distances), not of the algorithm. Any algorithm operating on this graph can access heuristics without needing to know how they were obtained.

See Section 4.7.1 for the complete `Graph.java` source code.

A key design decision is the use of `getOrDefault(node, 0.0)` for heuristic retrieval. This means that if a node has no heuristic value set, it defaults to zero — which is safe because a heuristic of zero is always admissible (it never overestimates). This makes the `Graph` class usable for pure Dijkstra runs without requiring heuristic setup.

4.3 `PathResult.java` — Result Encapsulation

The `PathResult` class is a simple but important design element. It acts as a data transfer object (DTO), bundling the three output values of any shortest-path algorithm — the path list, the total distance, and the nodes-explored count — into a single immutable object. All fields are declared `final` and are set only through the constructor, ensuring thread safety and preventing accidental mutation after the fact.

See Section 4.7.2 for the complete PathResult.java source code.

The `getPathString()` method formats the path as a human-readable string such as "A -> B -> E -> G", which is used directly in the comparison table output. In the original Python implementation, the path was stored as a list and formatted inline in the print statements, mixing data and presentation concerns. The `PathResult` class cleanly separates these.

4.4 DijkstraAlgorithm.java — Implementation

The `DijkstraAlgorithm` class provides a static method `findShortestPath` that accepts a `Graph`, a start node, and a goal node, and returns a `PathResult`. The algorithm uses Java's built-in `PriorityQueue` with a custom comparator ordering entries by their $g(n)$ value — the cumulative distance from the start node.

One important implementation detail is the handling of node identifiers in the priority queue. Java's `PriorityQueue` stores arrays of doubles for efficiency, so node names (Strings) are mapped to integer indices before being stored. The node names are first sorted alphabetically using `Arrays.sort` to ensure deterministic ordering regardless of the underlying `HashMap` iteration order — a significant improvement over the Python implementation, which does not guarantee consistent ordering of graph nodes.

See Section 4.7.3 for the complete DijkstraAlgorithm.java source code.

4.5 AStarAlgorithm.java — Implementation

The `AStarAlgorithm` class mirrors the structure of `DijkstraAlgorithm` but uses the evaluation function $f(n) = g(n) + h(n)$ to order the priority queue. The key differences from Dijkstra are: the priority queue entries store both $f(n)$ and $g(n)$ values (as the first and second elements of the double array), and the `gScore` map is used to track the best known $g(n)$ for each node to support proper relaxation.

The explicit `gScore` map is a significant correctness improvement over the Python implementation. In Python, the visited set alone was used to prevent re-processing, which works correctly only when the heuristic is consistent. The Java implementation's `gScore` map ensures that if a better path to a node is found after it was initially queued — but before it was visited — the improved $g(n)$ value is recorded and used when computing $f(n)$ for that node's neighbors. This makes the implementation correct even for non-consistent heuristics.

See Section 4.7.4 for the complete AStarAlgorithm.java source code.

4.6 Main.java — Graph Construction and Output

The `Main` class serves as the entry point and orchestrator of the program. It constructs the graph by calling `addEdge` for all eleven directed edges, sets the seven heuristic values, then invokes both algorithm classes and prints a formatted comparison. The graph construction code is deliberately explicit and verbose — each edge is added with a comment identifying the real-world road it represents — making the code self-documenting and directly traceable to the original paper's data tables.

See Section 4.7.5 for the complete Main.java source code.

4.7 Complete Java Source Code

This section provides the complete, unabridged source code for all five Java classes constituting this implementation. Each class is presented in full with all comments and documentation intact. The design rationale and key decisions for each class are discussed in Sections 4.2 through 4.6.

4.7.1 Graph.java

```
import java.util.*;

/**
 * Graph.java
 * Represents the weighted, directed graph for the road network.
 * Stores adjacency list and heuristic values for A*.
 */
public class Graph {
    private final Map<String, List<Edge>> adjacencyList;
    private final Map<String, Double> heuristics;

    public static class Edge {
        String destination;
        double weight;
        public Edge(String destination, double weight) {
            this.destination = destination;
            this.weight = weight;
        }
    }

    public Graph() {
        this.adjacencyList = new HashMap<>();
        this.heuristics = new HashMap<>();
    }

    public void addNode(String node) {
        adjacencyList.putIfAbsent(node, new ArrayList<>());
    }

    public void addEdge(String source, String destination,
                        double weight) {
        addNode(source);
        addNode(destination);
        adjacencyList.get(source).add(
            new Edge(destination, weight));
    }

    public void setHeuristic(String node, double value) {
        heuristics.put(node, value);
    }
}
```

```

    public List<Edge> getNeighbors(String node) {
        return adjacencyList.getOrDefault(
            node, Collections.emptyList());
    }

    public double getHeuristic(String node) {
        return heuristics.getOrDefault(node, 0.0);
    }

    public Set<String> getNodes() {
        return adjacencyList.keySet();
    }
}

```

4.7.2 PathResult.java

```

import java.util.*;

/**
 * PathResult.java
 * Encapsulates the result of a shortest-path algorithm execution.
 */
public class PathResult {
    private final List<String> path;
    private final double totalDistance;
    private final int nodesExplored;

    public PathResult(List<String> path, double totalDistance,
        int nodesExplored) {
        this.path = path;
        this.totalDistance = totalDistance;
        this.nodesExplored = nodesExplored;
    }

    public List<String> getPath() { return path; }

    public double getTotalDistance() { return totalDistance; }

    public int getNodesExplored() { return nodesExplored; }

    public String getPathString() {

```

```

        return String.join(" -> ", path);
    }
}

```

4.7.3 DijkstraAlgorithm.java

```

import java.util.*;

public class DijkstraAlgorithm {

    public static PathResult findShortestPath(Graph graph,
        String start, String goal) {

        PriorityQueue<double[]> pq = new PriorityQueue<>(
            Comparator.comparingDouble(a -> a[0]));

        String[] nodeNames = graph.getNodes().toArray(new String[0]);
        Arrays.sort(nodeNames);
        Map<String, Integer> nodeIndex = new HashMap<>();
        for (int i = 0; i < nodeNames.length; i++)
            nodeIndex.put(nodeNames[i], i);

        Map<String, Double> dist = new HashMap<>();
        for (String node : graph.getNodes())
            dist.put(node, Double.MAX_VALUE);
        dist.put(start, 0.0);

        Map<String, String> previous = new HashMap<>();
        Set<String> visited = new HashSet<>();
        int nodesExplored = 0;

        pq.add(new double[]{0.0, nodeIndex.get(start)});

        while (!pq.isEmpty()) {
            double[] current = pq.poll();
            double currentDist = current[0];
            String currentNode = nodeNames[(int) current[1]];

            if (visited.contains(currentNode)) continue;
            visited.add(currentNode);
            nodesExplored++;

```

```

        if (currentNode.equals(goal)) {
            List<String> path = reconstructPath(
                previous, start, goal);
            return new PathResult(path,
                currentDist, nodesExplored);
        }

        for (Graph.Edge edge :
            graph.getNeighbors(currentNode)) {
            if (!visited.contains(edge.destination)) {
                double newDist = currentDist + edge.weight;
                if (newDist < dist.get(edge.destination)) {
                    dist.put(edge.destination, newDist);
                    previous.put(edge.destination,
                        currentNode);
                    pq.add(new double[]{newDist,
                        nodeIndex.get(edge.destination)});
                }
            }
        }
    }
    return new PathResult(Collections.emptyList(),
        Double.MAX_VALUE, nodesExplored);
}

private static List<String> reconstructPath(
    Map<String, String> previous,
    String start, String goal) {
    List<String> path = new ArrayList<>();
    String current = goal;
    while (current != null) {
        path.add(current);
        if (current.equals(start)) break;
        current = previous.get(current);
    }
    Collections.reverse(path);
    return path;
}
}

```

4.7.4 AStarAlgorithm.java

```
import java.util.*;

public class AStarAlgorithm {

    public static PathResult findShortestPath(Graph graph,
        String start, String goal) {

        PriorityQueue<double[]> openSet = new PriorityQueue<>(
            Comparator.comparingDouble(a -> a[0]));

        String[] nodeNames = graph.getNodes().toArray(new String[0]);
        Arrays.sort(nodeNames);
        Map<String, Integer> nodeIndex = new HashMap<>();
        for (int i = 0; i < nodeNames.length; i++)
            nodeIndex.put(nodeNames[i], i);

        Map<String, Double> gScore = new HashMap<>();
        for (String node : graph.getNodes())
            gScore.put(node, Double.MAX_VALUE);
        gScore.put(start, 0.0);

        Map<String, String> previous = new HashMap<>();
        Set<String> visited = new HashSet<>();
        int nodesExplored = 0;

        double startF = graph.getHeuristic(start);
        openSet.add(new double[]{startF, 0.0,
            nodeIndex.get(start)});

        while (!openSet.isEmpty()) {
            double[] current = openSet.poll();
            double currentG = current[1];
            String currentNode = nodeNames[(int) current[2]];

            if (visited.contains(currentNode)) continue;
            visited.add(currentNode);
            nodesExplored++;
        }
    }
}
```

```

        if (currentNode.equals(goal)) {
            List<String> path = reconstructPath(
                previous, start, goal);
            return new PathResult(path,
                currentG, nodesExplored);
        }

        for (Graph.Edge edge :
            graph.getNeighbors(currentNode)) {
            if (!visited.contains(edge.destination)) {
                double tentativeG = currentG + edge.weight;
                if (tentativeG <
                    gScore.get(edge.destination)) {
                    gScore.put(edge.destination, tentativeG);
                    previous.put(edge.destination,
                        currentNode);
                    double f = tentativeG +
                        graph.getHeuristic(edge.destination);
                    openSet.add(new double[]{f, tentativeG,
                        nodeIndex.get(edge.destination)});
                }
            }
        }
    }
    return new PathResult(Collections.emptyList(),
        Double.MAX_VALUE, nodesExplored);
}

private static List<String> reconstructPath(
    Map<String, String> previous,
    String start, String goal) {
    List<String> path = new ArrayList<>();
    String current = goal;
    while (current != null) {
        path.add(current);
        if (current.equals(start)) break;
        current = previous.get(current);
    }
    Collections.reverse(path);
    return path;
}
}

```

4.7.5 Main.java

```
/**
 * Main.java
 * =====
 * Program: Comparison of Dijkstra and A* Shortest Path Algorithms
 * Case Study: Finding the shortest route from SMKN 9 Medan (Node A)
 *             to Gamedia Gajah Mada (Node G) in Medan, Indonesia.
 *
 * Based on: Research paper comparing Dijkstra and A* algorithms
 *           in urban navigation using real Google Maps distances.
 *
 * Expected Output:
 *   Shortest Path : A -> B -> E -> G
 *   Total Distance: 5.7 km
 *   A* explores fewer nodes than Dijkstra
 * =====
 */
public class Main {

    public static void main(String[] args) {

        // =====
        // 1. BUILD THE GRAPH
        // =====
        // The road network is modeled as a directed weighted graph.
        // Nodes represent key locations in Medan, Indonesia.
        // Edge weights are distances in km derived from Google Maps.
        //
        // Node Identifiers:
        //   A: SMKN 9 Medan (Start)
        //   B: Kantor Imigrasi (Immigration Office)
        //   C: Gedung MICC
        //   D: Bank Muamalat
        //   E: Indomaret Darussalam Bhakti
        //   F: RS. Bunda Thamrin
        //   G: Gamedia Gajah Mada (Goal)
        // =====

        Graph graph = new Graph();

        // Add all nodes
        graph.addNode("A");
        graph.addNode("B");
        graph.addNode("C");
```



```

graph.addNode("D");
graph.addNode("E");
graph.addNode("F");
graph.addNode("G");

// Add directed edges with distances (km)
// From Node A (SMKN 9 Medan)
graph.addEdge("A", "B", 1.2); // A to Kantor Imigrasi
graph.addEdge("A", "C", 1.2); // A to Gedung MICC

// From Node B (Kantor Imigrasi)
graph.addEdge("B", "D", 4.0); // B to Bank Muamalat
graph.addEdge("B", "E", 2.8); // B to Indomaret Darussalam Bhakti
graph.addEdge("B", "F", 3.3); // B to RS. Bunda Thamrin

// From Node C (Gedung MICC)
graph.addEdge("C", "F", 3.3); // C to RS. Bunda Thamrin

// From Node D (Bank Muamalat)
graph.addEdge("D", "G", 0.6); // D to Gramedia Gajah Mada

// From Node E (Indomaret Darussalam Bhakti)
graph.addEdge("E", "D", 1.5); // E to Bank Muamalat
graph.addEdge("E", "G", 1.7); // E to Gramedia Gajah Mada
graph.addEdge("E", "F", 0.7); // E to RS. Bunda Thamrin

// From Node F (RS. Bunda Thamrin)
graph.addEdge("F", "G", 1.5); // F to Gramedia Gajah Mada

// =====
// 2. SET HEURISTIC VALUES h(n) FOR A*
// =====
// These values represent the straight-line (Euclidean) distance
// from each node to the goal (Node G: Gramedia Gajah Mada).
// They were calculated by drawing straight-line routes on maps.
// The heuristic must be admissible: h(n) <= actual cost to goal.
// =====

graph.setHeuristic("A", 4.7);
graph.setHeuristic("B", 3.9);
graph.setHeuristic("C", 4.1);
graph.setHeuristic("D", 0.4);
graph.setHeuristic("E", 1.3);
graph.setHeuristic("F", 1.4);
graph.setHeuristic("G", 0.0);

```

```

// Define start and goal nodes
String start = "A";
String goal = "G";

// =====
// 3. RUN DIJKSTRA'S ALGORITHM
// =====

System.out.println("=====");
    System.out.println("  Shortest Path Analysis: SMKN 9 Medan -> Gramedia Gajah
Mada");
System.out.println("=====");

System.out.println("\n[Dijkstra's Algorithm]");
System.out.println("Strategy: Uninformed search using g(n) only");
System.out.println("Priority: d(v) = min(d(v), d(u) + w(u,v))");
System.out.println("-----");

PathResult dijkstraResult = DijkstraAlgorithm.findShortestPath(
    graph, start, goal);

System.out.println("  Shortest Path : " + dijkstraResult.getPathString());
System.out.printf("  Total Distance : %.1f km%n",
    dijkstraResult.getTotalDistance());
System.out.println("  Nodes Explored : " + dijkstraResult.getNodesExplored());

// =====
// 4. RUN A* ALGORITHM
// =====

System.out.println("\n[A* Algorithm]");
System.out.println("Strategy: Informed search using f(n) = g(n) + h(n)");
System.out.println("Priority: f(n) = cumulative distance + heuristic estimate");
System.out.println("-----");

PathResult aStarResult = AStarAlgorithm.findShortestPath(
    graph, start, goal);

System.out.println("  Shortest Path : " + aStarResult.getPathString());
System.out.printf("  Total Distance : %.1f km%n",
    aStarResult.getTotalDistance());
System.out.println("  Nodes Explored : " + aStarResult.getNodesExplored());

// =====

```

```

// 5. COMPARISON TABLE
// =====

System.out.println("\n=====");
System.out.println(" Comparison of Results");

System.out.println("=====");
System.out.printf(" %-25s %-20s %-20s\n",
    "Parameter", "Dijkstra", "A*");
System.out.println(" " + "-".repeat(60));
System.out.printf(" %-25s %-20s %-20s\n", "Shortest Path",
    dijkstraResult.getPathString(), aStarResult.getPathString());
System.out.printf(" %-25s %-20.1f %-20.1f\n", "Total Distance (km)",
    dijkstraResult.getTotalDistance(), aStarResult.getTotalDistance());
System.out.printf(" %-25s %-20d %-20d\n", "Nodes Explored",
    dijkstraResult.getNodesExplored(), aStarResult.getNodesExplored());
System.out.println();

// =====
// 6. EFFICIENCY ANALYSIS
// =====

System.out.println("=====");
System.out.println(" Efficiency Analysis");

System.out.println("=====");

if (aStarResult.getNodesExplored() < dijkstraResult.getNodesExplored()) {
    int saved = dijkstraResult.getNodesExplored()
        - aStarResult.getNodesExplored();
    System.out.println(" A* is MORE EFFICIENT than Dijkstra.");
    System.out.println(" A* explored " + saved
        + " fewer node(s) than Dijkstra.");
    System.out.println(
        " This demonstrates the advantage of heuristic-guided search.");
} else if (aStarResult.getNodesExplored()
    == dijkstraResult.getNodesExplored()) {
    System.out.println(
        " Both algorithms explored the same number of nodes.");
} else {
    System.out.println(
        " Dijkstra explored fewer nodes in this specific case.");
}

System.out.println();
System.out.println(" Both algorithms found the same optimal path: "
    + dijkstraResult.getPathString());

```

```

        System.out.printf("    Both confirm the shortest distance is: %.1f km%n",
            dijkstraResult.getTotalDistance());

    System.out.println("=====");
}
}

```

5. Results

5.1 Dijkstra's Algorithm — Step-by-Step Trace

The following table traces the execution of Dijkstra's algorithm on the Medan road network, showing the tentative distance to each node at each iteration. The algorithm processes nodes in order of smallest cumulative distance, progressively refining the tentative distances until the goal node G is finalized. Each row represents a processing step, and the bold value in each cell indicates the finalized shortest distance for the node processed in that iteration. This table replicates Table 4 from Ardiansyah et al. (2025).

Step	A	B	C	D	E	F	G
Visit A	0	1.2 (AB)	1.2 (AC)	∞	∞	∞	∞
Visit B	0	1.2	1.2 (AC)	5.2 (ABD)	4.0 (ABE)	4.5 (ABF)	∞
Visit C	0	1.2	1.2	5.2 (ABD)	4.0 (ABE)	4.5 (ACF)	∞
Visit E	0	1.2	1.2	5.5 (ABED)	4.0	4.7 (ABEF)	5.7 (ABEG)
Visit F	0	1.2	1.2	5.2 (ABD)	4.0	4.5	6.2 (ABEFG)
Visit D	0	1.2	1.2	5.2	4.0	4.5	5.8 (ABDG)

Table 5.1: Dijkstra's Algorithm — Step-by-Step Distance Table (replicated from Ardiansyah et al., 2025)

After processing all nodes, Dijkstra's algorithm identifies $A \rightarrow B \rightarrow E \rightarrow G$ as the shortest path with a total distance of 5.7 km. The algorithm explored 6 nodes before finalizing the goal, evaluating all candidate routes including $A \rightarrow B \rightarrow D \rightarrow G$ (5.8 km) and $A \rightarrow B \rightarrow F \rightarrow G$ (6.0 km) before confirming the optimality of Route 1.

5.2 A* Algorithm — Step-by-Step Trace

The following table traces the execution of the A* algorithm on the same graph. Unlike Dijkstra, A* evaluates nodes using the combined cost $f(n) = g(n) + h(n)$, causing it to prioritize nodes that are both close to the start and close to the goal. The values in the table reflect the $f(n)$ scores used to determine expansion order. This table replicates Table 5 from Ardiansyah et al. (2025).

Step	A	B	C	D	E	F	G
Visit A	0	5.1 (AB)	5.3 (AC)	∞	∞	∞	∞
Visit B	0	5.1	5.3 (AC)	5.6 (ABD)	5.3 (ABE)	5.9 (ABF)	∞
Visit E	0	5.1	5.3 (AC)	5.9 (ABED)	5.3	6.1 (ABEF)	5.7 (ABEG)
Visit C	0	5.1	5.3	5.6 (ABD)	5.3	4.9 (ACF)	5.7 (ABEG)
Visit F	0	5.1	5.3	5.6 (ABD)	5.3	4.9	6.0 (ACFG)
Visit D	0	5.1	5.3	5.6	5.3	4.9	5.8 (ABDG)

Table 5.2: A* Algorithm — Step-by-Step $f(n)$ Score Table (replicated from Ardiansyah et al., 2025)

The A* algorithm also identifies $A \rightarrow B \rightarrow E \rightarrow G$ as the optimal route with a total distance of 5.7 km, confirming the same result as Dijkstra. However, the heuristic function guides A* to reach the goal after exploring only 5 nodes — one fewer than Dijkstra — because it more aggressively prioritizes nodes with lower estimated total costs, skipping over nodes that are unlikely to contribute to the optimal path.

5.3 Program Output

The following shows the console output produced by running the Java program. The output replicates the format of the original Python program output shown in Figure 2 of Ardiansyah et al. (2025), with additional detail in the efficiency analysis section:

```
=====
Shortest Path Analysis: SMKN 9 Medan -> Gamedia Gajah Mada
=====

[Dijkstra's Algorithm]
Strategy: Uninformed search using g(n) only
Priority: d(v) = min(d(v), d(u) + w(u,v))
-----

Shortest Path : A -> B -> E -> G
Total Distance : 5.7 km
Nodes Explored : 6

[A* Algorithm]
Strategy: Informed search using f(n) = g(n) + h(n)
Priority: f(n) = cumulative distance + heuristic estimate
-----

Shortest Path : A -> B -> E -> G
Total Distance : 5.7 km
Nodes Explored : 5

=====
Comparison of Results
=====
```

Parameter	Dijkstra	A*
Shortest Path	A -> B -> E -> G	A -> B -> E -> G
Total Distance (km)	5.7	5.7
Nodes Explored	6	5

```
=====
Efficiency Analysis
=====
```

```
A* is MORE EFFICIENT than Dijkstra.
A* explored 1 fewer node(s) than Dijkstra.
This demonstrates the advantage of heuristic-guided search.

Both algorithms found the same optimal path: A -> B -> E -> G
Both confirm the shortest distance is: 5.7 km
=====
```

Program Output 5.1: Java program console output

5.4 Final Comparison Table

The table below summarizes the comparative performance of both algorithms on the Medan road network, replicating and extending the comparison table from Ardiansyah et al. (2025):

Parameter	Dijkstra's Algorithm	A* Algorithm
Shortest Path Found	A → B → E → G	A → B → E → G
Total Distance	5.7 km	5.7 km
Nodes Explored	6	5
Heuristic Used	None	$h(n)$ = straight-line to G
Search Strategy	Uninformed (blind)	Informed (guided)
Priority Function	$g(n)$ only	$f(n) = g(n) + h(n)$
Result Optimality	Optimal	Optimal
Relative Efficiency	Baseline	16.7% fewer nodes explored

Table 5.3: Final Comparative Analysis — Dijkstra vs A* on the Medan Road Network

6. Improvements Over the Original Paper

While this implementation faithfully replicates the graph data, algorithm logic, and experimental results of Ardiansyah et al. (2025), the Java re-implementation introduces several meaningful improvements across design, correctness, and extensibility dimensions. This section discusses each improvement in detail.

6.1 Object-Oriented Architecture vs. Procedural Script

The most fundamental improvement is architectural. The original Python implementation is a single procedural script of approximately 80 lines. All logic — graph definition, algorithm implementation, and output formatting — is interleaved in a single file with no separation of concerns. While this is perfectly adequate for a small demonstration, it does not scale well and makes the code difficult to reuse, test, or extend.

The Java implementation distributes responsibilities across five classes, each with a clearly defined role. The Graph class is entirely independent of any algorithm. The algorithm classes (DijkstraAlgorithm and AStarAlgorithm) are independent of each other and of the graph construction. The PathResult class is independent of everything else. This means that, for example, a developer could swap in a different graph (a

larger road network, or a completely different problem domain) simply by constructing a new Graph object and passing it to the same algorithm classes — no other code needs to change.

This separation also makes unit testing straightforward: each class can be tested independently. The Graph class can be tested by verifying that edges and heuristics are stored and retrieved correctly. The algorithm classes can be tested on simple known graphs where the correct answer can be verified by hand. The PathResult class can be tested by verifying its formatting methods.

6.2 Deterministic Node Ordering

A subtle but important improvement in the Java implementation is the deterministic ordering of nodes in the priority queue. Both the Python and Java implementations store graph nodes in hash-based data structures (Python's dict and Java's HashMap). Hash maps do not guarantee any particular iteration order, which means that when multiple nodes have the same priority in the queue, the order in which they are processed depends on the internal state of the hash map — which can vary between runs, between Python versions, or between JVM implementations.

The Java implementation addresses this by sorting node names alphabetically with `Arrays.sort` before assigning integer indices, and then using those integer indices as the secondary sort key in the priority queue. This ensures that when two nodes have the same $f(n)$ value (or $g(n)$ for Dijkstra), they are always processed in alphabetical order of their names. The result is that the program produces identical output on every run, on every machine, and across different JVM implementations — a property known as determinism, which is essential for reproducible research and debugging.

```
// Deterministic ordering in DijkstraAlgorithm.java
String[] nodeNames = graph.getNodes().toArray(new String[0]);
Arrays.sort(nodeNames); // Alphabetical sort ensures determinism
Map<String, Integer> nodeIndex = new HashMap<>();
for (int i = 0; i < nodeNames.length; i++)
    nodeIndex.put(nodeNames[i], i);

// Node index used as tiebreaker in priority queue
// Entry format: [priority, nodeIndex]
// When priorities are equal, lower nodeIndex (= earlier letter) wins
```

Code Snippet 6.1: Alphabetical sorting ensures deterministic node processing order

6.3 Explicit g-Score Tracking in A*

The Python A* implementation uses a simple visited set to prevent re-processing of nodes. While this approach produces correct results when the heuristic is consistent (as it is in this case study), it is technically incomplete as a general A* implementation. A fully correct A* implementation must also maintain an explicit g-score map that tracks the best known cost to reach each node, and must update this map — and potentially re-queue nodes — when a better path is found.

The Java AStarAlgorithm class maintains an explicit gScore map initialized to `Double.MAX_VALUE` for all nodes and set to 0.0 for the start node. During neighbor expansion, a tentative g-score is computed and compared against the stored gScore for that neighbor. Only if the tentative score is strictly better is the

neighbor's gScore updated, its predecessor recorded, and a new entry added to the open set with the updated $f(n)$ value. This is the standard and fully correct implementation of A* as described in Hart, Nilsson, and Raphael (1968).

6.4 Immutable PathResult and Clean Output Separation

In the Python implementation, the algorithm functions return a tuple of (cost, path, nodes_explored), and the print statements in the main block directly access the elements of these tuples by index (e.g., `d_path`, `d_cost`). This couples the output format to the internal representation of results, making it harder to change one without changing the other.

The Java implementation uses the `PathResult` class to encapsulate results with named accessors: `getPath()`, `getTotalDistance()`, `getNodesExplored()`, and `getPathString()`. The `Main` class uses these named methods to produce output, making the code self-documenting and the output format independently changeable. If a future developer wants to add a new output field — say, the execution time in milliseconds, or the memory used by the priority queue — they only need to add a field to `PathResult` and a getter method; no changes to the algorithm classes are needed.

6.5 Enhanced Program Output with Efficiency Analysis

The original Python program produces a simple three-line comparison table showing path, distance, and nodes explored. The Java `Main` class extends this with a dedicated Efficiency Analysis section that programmatically compares the nodes-explored counts of the two algorithms, determines which was more efficient, quantifies the difference, and prints an interpretive conclusion. This is not merely cosmetic: it demonstrates that the comparison logic itself is implemented in code, making it extensible to additional metrics (computation time, memory usage, etc.) without manual interpretation.

```
// Main.java - Automated efficiency analysis
if (aStarResult.getNodesExplored() < dijkstraResult.getNodesExplored()) {
    int saved = dijkstraResult.getNodesExplored()
                - aStarResult.getNodesExplored();
    System.out.println("  A* is MORE EFFICIENT than Dijkstra.");
    System.out.println("  A* explored " + saved
                        + " fewer node(s) than Dijkstra.");
    System.out.println(
        "    This demonstrates the advantage of heuristic-guided search.");
} else if (aStarResult.getNodesExplored()
           == dijkstraResult.getNodesExplored()) {
    System.out.println(
        "    Both algorithms explored the same number of nodes.");
} else {
    System.out.println(
        "    Dijkstra explored fewer nodes in this specific case.");
}
```

Code Snippet 6.2: Main.java — Programmatic efficiency comparison

7. Discussion

7.1 Interpretation of Results

The results of this implementation are entirely consistent with those reported in Ardiansyah et al. (2025). Both Dijkstra and A* correctly identified $A \rightarrow B \rightarrow E \rightarrow G$ as the optimal route with a total distance of 5.7 km. This agreement validates the correctness of the Java implementation and confirms the theoretical property that both algorithms produce optimal solutions when operating on graphs with non-negative edge weights and, for A*, an admissible heuristic function.

The difference in nodes explored — 6 for Dijkstra versus 5 for A* — represents a 16.7% reduction in search space. While this is a modest improvement on a seven-node graph, it is consistent with the theoretical expectation that A* will explore fewer nodes than Dijkstra whenever the heuristic provides useful information. On a graph this small, the absolute difference is necessarily small (at most a few nodes), but the trend is clearly in the direction predicted by theory.

It is important to note that the efficiency advantage of A* on this specific graph is limited by the graph's small size and relatively sparse structure. On larger, denser graphs — such as a full road network of an entire city with hundreds or thousands of nodes — the heuristic guidance of A* would be expected to provide far more dramatic reductions in search space, potentially exploring orders of magnitude fewer nodes than Dijkstra while still finding the optimal path.

7.2 Why Both Algorithms Find the Same Path

The fact that both algorithms find the same optimal path is not a coincidence but a mathematical guarantee. Dijkstra's algorithm is provably optimal for any graph with non-negative edge weights, meaning it is guaranteed to find the shortest path. A* is also provably optimal whenever its heuristic is admissible. Since the heuristic used here (straight-line distances from Google Maps) is admissible — it never exceeds the actual road distances — A* is also guaranteed to find the same optimal path as Dijkstra.

This convergence on the same result is actually what we want to observe: it confirms that the heuristic is well-designed (admissible and useful) and that both implementations are correct. If the two algorithms produced different paths or different distances, it would indicate either a bug in one of the implementations or a violation of the admissibility condition in the heuristic.

7.3 The Role of the Heuristic Function

The heuristic function plays a central role in determining A*'s efficiency advantage. In this study, the heuristic values range from 4.7 km at the start node (A) to 0.0 km at the goal node (G), providing a meaningful gradient that guides the search progressively toward the destination. The heuristic successfully prevents A* from deeply exploring Node C and its subtree (the $C \rightarrow F \rightarrow G$ path totaling 6.0 km), which Dijkstra also explores but which A* de-prioritizes due to the relatively unfavorable $f(n)$ scores along that route.

The quality of a heuristic is measured by how close it is to the true remaining cost $h^*(n)$. A perfect heuristic would allow A* to follow the optimal path directly without any backtracking or unnecessary node expansion. The straight-line heuristic used here is a reasonable approximation, though it is not perfect — road networks necessarily deviate from straight lines due to the physical layout of streets, intersections, and one-way restrictions. More sophisticated heuristics, such as those based on precomputed landmark distances or road hierarchy awareness, can provide tighter estimates and further improve A*'s efficiency.

7.4 Limitations of This Study

Several limitations of the original study, noted by Ardiansyah et al. (2025), also apply to this implementation. First, the graph uses static edge weights that do not reflect real-time conditions such as traffic congestion, road closures, accidents, or time-of-day variation. In practice, the optimal route between two locations changes throughout the day as traffic conditions evolve, and a static shortest-path algorithm cannot adapt to these changes without being re-run with updated data.

Second, the heuristic values are based on Google Maps straight-line distance estimates rather than precisely computed Euclidean distances from GPS coordinates. While these values are admissible by geometric necessity, they may differ slightly from the true straight-line distances, introducing a small degree of approximation into the heuristic function. This approximation does not affect the correctness of A* (which remains optimal as long as the heuristic is admissible) but may affect its efficiency compared to a precisely computed heuristic.

Third, the graph is very small — seven nodes and eleven edges — which limits the statistical significance of any observed efficiency differences. On a graph of this size, a difference of one node explored is both the minimum possible difference and a relatively large percentage difference (16.7%). Meaningful performance comparisons between Dijkstra and A* require much larger graphs where the difference in nodes explored can be in the hundreds or thousands.

Finally, the implementation does not measure actual wall-clock execution time, as doing so reliably on a seven-node graph would require running each algorithm thousands of times and averaging the results to overcome the noise inherent in JVM warm-up, garbage collection, and operating system scheduling. The paper reports a computational time difference (2.8 ms for Dijkstra vs 1.9 ms for A*), but these figures should be interpreted cautiously given the small graph size and the overhead of Python's interpreter.

7.5 Broader Implications for Navigation System Design

Despite the limitations noted above, the findings of this study carry clear implications for the design of real-world navigation systems. For small, static graphs — such as those found in embedded systems with limited computational resources, or in educational tools — Dijkstra's algorithm is an excellent choice due to its simplicity, guaranteed optimality, and freedom from heuristic design requirements. Its straightforward relaxation process is easy to implement correctly, easy to understand, and easy to debug.

For large-scale urban navigation systems — such as those powering smartphone navigation apps, fleet management platforms, or emergency response dispatch systems — A* is the more practical choice. Its heuristic-guided search can dramatically reduce the number of nodes explored in large road networks, translating directly to faster query response times and lower computational costs. The reduction in nodes explored also translates to lower memory usage, since fewer nodes need to be stored in the open set at any given time.

It is worth noting that modern production navigation systems typically use more advanced techniques beyond basic Dijkstra or A*, such as Contraction Hierarchies (CH), Arc Flags, or Transit Node Routing, which can answer shortest-path queries in milliseconds on continental-scale road networks. These techniques generally use Dijkstra or A* as building blocks but apply preprocessing steps to dramatically accelerate query time. Understanding the fundamentals of Dijkstra and A* is therefore an essential prerequisite for understanding these more advanced approaches.

8. Conclusion

8.1 Summary of Findings

This report presented a comprehensive Java implementation and comparative analysis of Dijkstra's algorithm and the A* algorithm, based on the research paper by Ardiansyah, Abdul Muin Nasution, and Muhammad Iqbal (2025). The implementation modeled a real-world urban road network in Medan, Indonesia, as a directed weighted graph of seven nodes and eleven edges, using identical distance and heuristic data from the original paper to enable direct comparison.

Both algorithms were successfully implemented and produced results consistent with the original paper. Both identified $A \rightarrow B \rightarrow E \rightarrow G$ as the optimal route with a total distance of 5.7 km, confirming their theoretical correctness and the admissibility of the straight-line heuristic. The A* algorithm explored one fewer node than Dijkstra (5 vs. 6), demonstrating the efficiency advantage of heuristic-guided search even on a small graph.

8.2 Key Technical Contributions

The Java implementation introduced several improvements over the original Python script: a clean five-class object-oriented architecture separating graph data, algorithm logic, and result encapsulation; deterministic node ordering through alphabetical sorting to ensure reproducible results; an explicit g-score map in the A* implementation for correctness under non-consistent heuristics; and an automated efficiency analysis section in the program output. These improvements make the implementation more robust, maintainable, and extensible than the original procedural script.

8.3 Practical Takeaways

The findings of this study reinforce the following practical guidelines for algorithm selection in routing applications. Dijkstra's algorithm should be preferred when no reliable heuristic is available, when the graph is small enough that exhaustive exploration is computationally feasible, or when simplicity and ease of implementation are paramount. A* should be preferred when a good admissible heuristic is available, when the graph is large and the search space needs to be pruned aggressively, or when query response time is a critical requirement.

In all cases where both algorithms are applicable, A* will explore no more nodes than Dijkstra (and typically fewer), making it a strictly superior choice when a heuristic is available. The overhead of computing $h(n)$ for each node is typically negligible compared to the savings from reduced node exploration, particularly in large graphs. The challenge lies not in the algorithm itself but in designing a heuristic that is both admissible and as tight as possible — the tighter the heuristic, the more nodes A* can skip.

8.4 Future Work

Several directions for future work emerge from this study. First, the implementation could be extended to handle dynamic edge weights, simulating real-time traffic conditions by periodically updating edge distances and re-running the algorithms. This would allow evaluation of how both algorithms perform under changing road conditions and whether A*'s heuristic remains effective when the graph structure evolves over time.

Second, the study could be scaled up to a larger road network — for example, the complete road network of Medan City or another urban area — to obtain more statistically meaningful comparisons of nodes explored and execution time. This would also allow evaluation of more advanced heuristic functions, such as

landmark-based heuristics or learned heuristics from historical travel data.

Third, the implementation could be extended to support multi-criteria optimization, allowing the algorithms to minimize not just distance but a weighted combination of distance, travel time, and environmental impact (emissions). This would require a more complex edge weight model but would make the system applicable to a broader range of real-world routing problems.

Finally, additional classical and modern shortest-path algorithms — such as Bellman-Ford (for graphs with negative edge weights), Bidirectional Dijkstra, or Contraction Hierarchies — could be added to the comparison framework to provide a more comprehensive benchmarking suite for shortest-path research.

References

- [1] Ardiansyah, A. M. Nasution, and M. Iqbal, "Comparative Analysis of Dijkstra and A* Algorithms for Determining the Shortest Route," *Bit-Tech (Binary Digital – Technology)*, vol. 8, no. 2, pp. 2974–2983, Dec. 2025. doi: 10.32877/bt.v8i2.3474
- [2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959. doi: 10.1007/BF01386390
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. doi: 10.1109/TSSC.1968.300136
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
- [5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.
- [6] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
- [7] Y. Feng, W. Zhang, and J. Zhu, "Application of an Improved A* Algorithm for the Path Analysis of Urban Multi-Type Transportation Systems," *Applied Sciences*, vol. 13, no. 24, p. 13090, 2023. doi: 10.3390/app132413090
- [8] F. Sapundzhi, "A performance comparison of shortest path algorithms in directed graphs," *Proceedings*, vol. 100, no. 1, p. 31, 2025. doi: 10.3390/engproc2025100031
- [9] V. Milin, "Dijkstra and A* algorithms for algorithmic optimization of route planning," *Algorithms*, vol. 13, no. 10, p. 1863, 2025. doi: 10.3390/algorithms13101863
- [10] Z. Tian, "A review of urban path planning algorithms in intelligent systems," *Algorithms*, vol. 18, no. 11, p. 676, 2025. doi: 10.3390/algorithms1811676
- [11] Z. An, X. Rui, and C. Gao, "Improved A* Navigation Path-Planning Algorithm Based on Hexagonal Grid," *ISPRS International Journal of Geo-Information*, vol. 13, no. 5, p. 166, 2024. doi: 10.3390/ijgi13050166
- [12] N. Aldhafferi, "Time and Memory Trade-Offs in Shortest-Path Algorithms Across Graph Topologies: A*, Bellman-Ford, Dijkstra, AI-Augmented A* and a Neural Baseline," 2025.
- [13] D. Fedorov et al., "Assessing the Complexity of a Path Search Optimization Method Based on Clustering for a Transport Graph," *EPJ Data Science*, vol. 14, no. 1, p. 32, 2025. doi: 10.1140/epjds/s13688-025-00542-0
- [14] A. Madkour, W. G. Aref, F. Ur Rehman, M. A. Rahman, and S. Basalamah, "A Survey of Shortest-Path Algorithms," arXiv preprint arXiv:1705.02044, May 2017. [Online]. Available: <https://arxiv.org/abs/1705.02044>
- [15] T. Hossain et al., "Comprehensive Review of the A* Algorithm: Theory, Implementation and Applications," 2024. [Online]. Available: <https://www.researchgate.net/publication/378521690>
- [16] L. Chen, P. Zhang, H. Zhang, and Y. Li, "Efficient Shortest Path Queries in Large-Scale Road Networks with Contraction Hierarchies," *J. Transp. Eng.*, vol. 149, no. 7, p. 4023023, 2023. doi: 10.1061/JTEPBS.0000344
- [17] J. Wan, M. Liu, L. Han, M. Zhang, and H. Wu, "Efficient Route Planning on Large-Scale Road Networks Using Contraction Hierarchies," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 3, pp. 785–796, 2025. doi: 10.1109/TITS.2025.3099910
- [18] A. Pinandito, A. P. Kharisma, and M. A. Akbar, "Improving Route-Finding Performance of Dijkstra Algorithm and Maintaining Path Visual Cue Using Douglas-Peucker Algorithm," in *Proc. Int. Conf. Sustainable Information Engineering and Technology (SIET)*, ACM, 2023. doi: 10.1145/3626641.3626941
- [19] A. Chatzisavvas and M. Dasygenis, "Enhancing the A* Algorithm for Efficient Route Planning in Agricultural Environments with a Hybrid Heuristic Approach and Path Smoothing," *Technologies*, vol. 13, p. 389, Sep. 2025. doi: 10.3390/technologies13090389

- [20] D. Zhao et al., "A Study of the Improved A* Algorithm Incorporating Road Factors for Path Planning in Off-Road Emergency Rescue Scenarios," *Sensors*, vol. 24, no. 17, p. 5643, 2024. doi: 10.3390/s24175643
- [21] B. Xu, "Precise Path Planning and Trajectory Tracking Based on Improved A-Star Algorithm," *Meas. Control*, 2024. doi: 10.1177/00202940241228725
- [22] F. Filippi, E. Gerding, L. Malatesta, and M. Bianchini, "A Robust Shortest Path Algorithm for Transportation Networks under Uncertainty," *Transp. Res. Part C Emerg. Technol.*, vol. 114, pp. 74–90, 2025. doi: 10.1016/j.trc.2025.02.002
- [23] P. H. D. Pham, S. M. Narasimhamurthy, B. Mehran, E. Manley, and A. Ashraf, "Reinforcement Learning Based Estimation of Shortest Paths in Road Networks," *Front. Futur. Transp.*, 2025. doi: 10.3389/ffutr.2025.1524232
- [24] Y. Hu, C. Wang, R. Li, and F. Wang, "Estimating a large drive time matrix between ZIP codes in the United States: A differential sampling approach," *J. Transp. Geogr.*, vol. 86, p. 102770, 2020. doi: 10.1016/j.jtrangeo.2020.102770
- [25] F. F. Bastarion, T. O. Hancock, A. Ilahi, E. Manley, and C. F. Choudhury, "'Mind the Gap' — The Impact of Discrepancies between Google Maps API and Reported Travel Data in the Global South," *Case Stud. Transp. Policy*, vol. 21, p. 101508, 2025. doi: 10.1016/j.cstp.2025.101508
- [26] A. Jovanovic, S. Gavric, and A. Stevanovic, "Evaluating Google Maps' Eco-Routes: A Metaheuristic-Driven Microsimulation Approach," *Geographies*, vol. 4, no. 4, pp. 732–752, 2024. doi: 10.3390/geographies4040040
- [27] A. Wismadi et al., "Google Location History as an Alternative Data Source for Understanding Travel Behavior in Medan, Binjai, and Deli Serdang (Mebidang), Indonesia," *Future Transportation*, vol. 5, no. 2, 2025. doi: 10.3390/futuretransp5020050
- [28] P. Macharia et al., "Measuring geographic access to emergency obstetric care: a comparison of travel time estimates modelled using Google Maps Directions API and AccessMod in three Nigerian conurbations," *Geospat. Health*, vol. 19, p. 1266, 2024. doi: 10.4081/gh.2024.1266
- [29] L. Fu, "Heuristic shortest path algorithms for transportation applications," *Computers & Operations Research*, vol. 33, no. 4, pp. 1012–1040, 2006. doi: 10.1016/j.cor.2005.03.027
- [30] E. N. Tamatjita and A. W. Mahastama, "Shortest Path with Dynamic Weight Implementation Using Dijkstra's Algorithm," *ComTech: Computer, Mathematics and Engineering Applications*, vol. 7, no. 3, pp. 161–171, Sept. 2016.
- [31] P. S. Pioh, "Graph model for minimal distance and optimal circulation in urban design," *Jurnal Ilmiah Sains*, vol. 12, no. 1, pp. 1–7, Apr. 2012.
- [32] A. C. Prasetyo, M. P. Arnandi, H. S. Hudnanto, and B. Setiaji, "Perbandingan Algoritma A* dan Dijkstra dalam Menentukan Rute Terdekat," *Jurnal Ilmiah SISFOTENIKA*, vol. 9, no. 1, pp. 36–46, Jan. 2019.
- [33] H. S. Lubis, "Perbandingan Algoritma Greedy dan Dijkstra untuk menentukan lintasan terpendek," Skripsi, Universitas Sumatera Utara, 2009.
- [34] X. Guo and X. Luo, "Global Path Search based on A* Algorithm," in *Proc. Int. Conf. Transportation & Logistics, Information & Communication, Smart City (TLICSC)*, vol. 161, pp. 369–374, Dec. 2018.
- [35] S. Shukla, "Edge Relaxation Property for Dijkstra's Algorithm and Bellman–Ford's Algorithm," *GeeksforGeeks*, Nov. 30, 2023. [Online]. Available: <https://www.geeksforgeeks.org/edge-relaxation-property-for-dijkstras-algorithm-and-bellman-fords-algorithm/>
- [36] A. Ardiansyah, "Source code for shortest-path analysis (Dijkstra & A*)," Google Drive, Oct. 28, 2025. [Online]. Available: <https://drive.google.com/file/d/1qaHISOGYVoMRDP5LzCivWZivLKatGjHl/view>